重构

代码重复（Duplicated Code）

问题：相同或高度相似的逻辑出现在多个地方，比如多个类里都有相同的验证逻辑，或者多个方法都在做同样的数据转换。

重构方式

- 提取公共函数 / 公共类
- 合并重复逻辑

可用设计模式

- **Strategy**：把“变化的算法”提取为策略类，用多态替代重复实现
- **Decorator**：重复的是“附加功能”时，把附加功能拆成装饰器

方法过长（Long Method）

问题：一个方法干了太多事情，比如一个 `processOrder()` 方法里既验证数据、又计算价格、又发邮件、又写日志，超过几十行，难以理解和维护。

重构方式

- Extract Method（拆方法）
- 让每个方法只做一件事

可用设计模式（特定情况）

- **Strategy**：当方法中有大段算法逻辑时，提取为策略
- **Command**：当方法包含完整操作流程时，封装为命令对象

类接口抽象层次不一致

问题：接口中混合高层操作和底层细节，比如一个类既有 `saveUser()` 这种高层方法，又有 `openConnection()` `executeSQL()` 这种底层方法暴露给外部。

重构方式

- 重新设计接口
- 隐藏实现细节，只暴露高层语义

可用设计模式

- **Adapter**：用适配器隔离底层细节，对外提供统一高层接口
- **Decorator**：把底层细节操作封装为装饰器，保持核心接口简洁

循环过长 / 嵌套过深

问题：循环体内逻辑太复杂，或者三四层 `for` 嵌套 `if`，比如 `for (订单) { for (商品) { if (...) { if (...) { ... }}}}`，很难追踪执行路径。

重构方式

- 提取循环为独立方法
- 拆分嵌套逻辑

可用设计模式（特定情况）

- **Iterator**：当循环本质是"遍历集合结构"时，用迭代器封装遍历逻辑

public 数据成员

问题：破坏封装，外部可直接修改内部状态，导致维护风险大。

重构方式

- 改为 private
- 提供必要的访问方法（getter / setter 或行为方法）

可用设计模式

- **Observer**：当多处需要感知状态变化时，用通知机制代替直接访问，把数据改为 private 后，在 setter 里调用 `notifyObservers()`，其他类作为观察者自动收到通知

修改需要并行改多个类（Parallel Change）

问题：一次需求改动，需要同时修改多个类，比如加一个用户字段，要改 `UserModel`、`UserDAO`、`UserService`、`UserController` 四个类。

可用设计模式

- **Observer**
  - 把变化集中到 Subject
  - 其他类作为 Observer 自动响应变化
- **Abstract Factory / DI**
  - 把"一整套一起变化的对象"集中创建，避免散落在各处修改

基本类型滥用（Primitive Obsession）

问题：用基本类型表示复杂业务概念，比如用 `String` 表示电话号码（没有格式校验）、用 `int` 表示金额（没有货币单位和精度处理）。

重构方式

- 为概念新建一个类（比如 money 类）
- 用这个类替代基本类型（比如 `salary = new money(3000)`）

类没干啥事（Lazy Class）

问题：类职责极少，没有存在价值，比如一个类只有一个简单的 getter/setter，或者只是把调用转发给另一个类，增加不必要的复杂度。

重构方式

- 删除类
- 或合并进相关类
- 如果是过度设计遗留，评估是否真的需要这层抽象

用注释解释难懂代码

问题：代码本身不可读，需要注释才能理解逻辑，比如 `if (status == 3 && type == 2) // 已支付且为会员订单`，应该用方法名 `isPaidMemberOrder()` 代替。

重构方式

- 重写代码
- 拆方法、改命名
- 让代码"自解释"
- 用有意义的变量名和函数名替代注释

使用全局变量

问题：隐式依赖，任何地方都能改，难追踪修改来源，比如一个全局 `currentUser` 被十几个模块直接修改，出了 bug 不知道是哪里改坏的，也难以单元测试。

重构方式

- 封装到对象里
- 通过参数或构造函数传递
- 使用依赖注入

"将来可能用到"的代码

问题：臆测需求，增加复杂度，违反 YAGNI 原则，比如现在只支持单一支付方式，却提前写了"支付策略抽象层"以备将来扩展，但实际可能永远用不到。

重构方式

- 删除代码（You Aren't Gonna Need It）
- 只实现当前需要的功能

相关数据未封装成类

问题：一组数据总是一起使用，却不属于同一个类（比如 `int year, month, day`）。

重构方式

- Extract Class
- 把数据和操作放进同一个类

- 用对象替代数据团（Data Clumps）

类内聚性差（职责过多）

问题：一个类承担多个不相关职责，比如一个 `User` 类既管理用户数据、又负责发邮件、又负责写日志、又负责权限校验，违反单一职责原则。

重构方式

- 拆分类，按职责划分

可用设计模式

- **Command**：把"操作 / 行为"从类中拆出去，封装成命令对象，类只负责数据和基本操作，复杂业务流程交给命令对象执行
- **Composite**：当类同时处理整体与局部逻辑时，用组合模式把容器节点和叶子节点分离，每个类只负责自己层次的职责

方法更依赖别的类（Feature Envy）

问题：方法大量操作另一个类的数据，比如 `OrderService.calculatePrice()` 里面全是 `order.getItem().getPrice()` `order.getDiscount()` 等，说明这个方法应该放在 `Order` 类里。

重构方式

- Move Method（把方法移到更合适的类）

可用设计模式

- **Visitor**：当对一组稳定结构反复添加新操作时，不要把操作方法都塞到元素类里，而是用访问者模式把操作外置，元素类只提供 `accept(visitor)` 接口

参数列表过长

问题：方法抽象失败，理解困难，调用繁琐易错，比如 `createUser(name, age, phone, email, address, city, zipCode, country)` 有八个参数，调用时容易传错顺序。

重构方式

- 封装参数对象（Parameter Object）
- 让对象自己提供所需信息
- 拆分方法职责

类内改动总是局部发生

问题：类中存在多个独立变化点，比如一个类里有多个 `if-else` 判断不同业务场景，每次加新场景只改其中一部分代码，说明这些变化点应该独立出去。

可用设计模式

- **Strategy**：把变化点拆成独立策略类，用组合替代条件判断，每个业务场景对应一个策略实现，类只持有策略接口，运行时切换策略对象即可，不需要修改原类

继承体系需要并行修改

问题：每加一个子类，另一棵继承树也要加，比如加一个 `MacButton`，就必须同时加 `MacTextBox`、`MacCheckbox`，两个继承体系被强制绑定。

可用设计模式

- **Abstract Factory**：把成套创建的逻辑集中到工厂里，加新平台时只需新增一个 `MacFactory` 实现 `createButton()` `createTextBox()` 等方法，客户端代码完全不改
- **Strategy**：用组合替代继承，比如 `Button` 不按平台继承，而是持有一个 `RenderStrategy`，切换平台只需换策略对象，避免 `WindowsButton` `MacButton` `LinuxButton` 继承爆炸

多处 case / switch 需要并行修改

问题：新增类型要改很多 `switch`，比如加一个新的订单状态，要改显示逻辑、计算逻辑、通知逻辑里的三个 `switch`，容易漏改。

可用设计模式

- **Strategy**：把每个分支对应的算法提取为策略类，用一个 `Map<类型, Strategy>` 或工厂方法获取策略，新增类型只需加新策略类，不需要改所有 `switch`
- **Command**：把每个分支的行为封装成命令对象，用 `CommandFactory` 根据类型返回对应命令，执行时统一调 `command.execute()`，新增类型只需加命令类
- **Visitor**：当类型结构稳定（如 AST 节点类型固定）但操作经常变化时，把操作封装到访问者里，加新操作只需加新 Visitor，不需要改每个类型的 `switch`

类之间过度亲密

问题：类之间依赖过深，封装被破坏，比如 `ClassA` 直接访问 `ClassB` 的私有字段（通过反射或友元），或者两个类互相调用对方的内部方法。

可用设计模式

- **Adapter**：用适配器隔离不合适的接口，两个类不直接依赖，而是通过适配器转换，让 `ClassA` 只依赖适配器接口，适配器负责转换调用 `ClassB`

命名很差

问题：类名 / 方法名无法表达意图，增加理解成本，比如方法叫 `doIt()` `process()` `handle()`，或者变量叫 `data` `temp` `obj`，完全不知道干什么的。

重构方式

- Rename（重命名）
- 使用能反映业务含义的名字
- 避免缩写和模糊命名

子类只用父类很小一部分

问题：继承关系本身不合理，比如 `Stack` 继承 `ArrayList`，但实际只用了 `add()` 和 `remove()`，其他几十个方法都用不上，还会暴露不该有的接口。

可用设计模式

- **Strategy**：用组合替代继承，比如 `Stack` 内部持有一个 `List` 对象，只暴露自己需要的 `push()`/`pop()` 方法，而不是继承暴露所有方法
- **Decorator**：如果只是想扩展部分功能，用装饰器包装原对象即可，比如给 `Reader` 加缓冲用 `BufferedReader` 装饰，不需要继承整个体系

方法调用前后大量 setup / teardown

问题：调用协议复杂且重复，比如每次调用数据库操作都要 `openConnection()` → `beginTransaction()` → 执行 → `commit()` → `closeConnection()`，到处都是这套流程。

可用设计模式

- **Command**：把整个调用过程（setup + 业务逻辑 + teardown）封装为命令对象，命令的 `execute()` 内部自动处理前后流程，调用方只需 `command.execute()` 即可
- **Decorator**：把前后逻辑作为可叠加的装饰层，比如 `TransactionDecorator` 自动加事务、`LoggingDecorator` 自动加日志，调用方只关心核心业务